

Attention Mechanisms in Transformer Networks

Page 1 of 2

The Attention mechanism is the defining innovation behind the Transformer architecture, introduced by Vaswani et al. in the landmark 2017 paper 'Attention Is All You Need'. Prior to Transformers, sequence transduction models relied heavily on recurrent neural networks (RNNs) or convolutional neural networks (CNNs). RNNs process tokens sequentially, creating a bottleneck for long-range dependencies and preventing parallelization during training. Attention bypasses this by allowing direct connection between any two tokens in a sequence, regardless of distance.

The mathematical core of the Transformer is Scaled Dot-Product Attention. Given an input sequence, the model computes three matrices: Queries (Q), Keys (K), and Values (V) via linear projections of token embeddings. The attention weights are calculated by taking the dot product of the query matrix with the transpose of the key matrix. This score is scaled by the square root of the dimension of the keys (d_k) to prevent gradients from vanishing during softmax. The formula is written as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

Attention Mechanisms in Transformer Networks

Page 2 of 2

Multi-Head Attention expands upon single scaled dot-product attention. Instead of performing a single attention function, the model projects queries, keys, and values 'h' times with different, learned linear projections to lower dimensions. On each of these projected versions, attention is performed in parallel, yielding output values. These outputs are concatenated and projected back to the original dimension.

This multi-head approach allows the model to jointly attend to information from different representation subspaces at different positions. For example, one head might focus on grammatical relationships (like subject-verb agreement), while another focuses on semantic associations or temporal order. In self-attention layers, all queries, keys, and values come from the same previous layer, allowing each position in the encoder to attend to all positions in the previous layer of the encoder.